

Plantilla multi-formato para investigación aplicada en buenas prácticas de desarrollo de software

Keiner Andres Cano Narvaez
Servicio Nacional de Aprendizaje (SENA)
Neiva, Colombia
ckeinercano@gmail.com

Jesús Ariel González Bonilla
Servicio Nacional de Aprendizaje (SENA)
Neiva, Colombia
instructor@ejemplo.edu

Resumen

1. Resumen

Este artículo presenta un análisis documental acerca de la integración de paradigmas, metodologías y arquitecturas que fortalecen el desarrollo de software contemporáneo. Entre los conceptos revisados se encuentran la Programación Orientada a Objetos (POO) [Izquierdo 2008], el patrón Modelo-Vista-Controlador (MVC) [Camarena Sagredo et al. 2012], los frameworks Node.js y Spring Boot [Haro et al. 2019], las metodologías ágiles como Scrum [Trigás Gallego 2012], y el papel de las bases de datos en los sistemas modernos [Pisco Gómez et al. 2017]. Se empleó un enfoque cualitativo de investigación documental, revisando artículos académicos y aplicando sus hallazgos a un proyecto formativo. Los resultados evidencian que la adecuada articulación entre arquitecturas, metodologías y paradigmas aporta organización, mantenibilidad y escalabilidad a los proyectos. Asimismo, metodologías ágiles como Scrum [3] fortalecen la comunicación, el trabajo colaborativo y la entrega incremental de valor.

Muchas de las investigaciones consultadas fueron aplicadas directamente en el proyecto, como el uso de Scrum [Trigás Gallego 2012], que permitió agilizar el desarrollo mediante la organización del equipo de trabajo en Sprints, facilitando el orden de tareas y la obtención de resultados funcionales en cada iteración. Este marco de trabajo resultó especialmente útil en un entorno donde se esperaban avances significativos en tiempos reducidos.

En cuanto a las bases de datos, se identificó la importancia de modelos robustos y escalables que puedan adaptarse a la evolución de las aplicaciones. El uso de sistemas NoSQL se plantea como una alternativa en proyectos que, como el nuestro, requieren integrar el Frontend con una API capaz de gestionar consultas y mantener la coherencia de los datos [Pisco Gómez et al. 2017].

El patrón Modelo-Vista-Controlador (MVC) [Camarena Sagredo et al. 2012] también se consolidó como una herramienta clave, ya que permite separar las responsabilidades del software: el Modelo (datos y lógica de negocio), la Vista (interfaz de usuario) y el Controlador (flujo de eventos y coordinación). Esta división favorece la mantenibilidad y la escalabilidad del sistema.

Por su parte, la Programación Orientada a Objetos (POO) [Izquierdo 2008] fue aplicada como base en el desarrollo con Java, garantizando un nivel adecuado de modularidad, reutilización y adaptabilidad del software. Sus principios —como la herencia, el encapsulamiento y el polimorfismo— resultaron esenciales para mantener la coherencia y facilitar modificaciones posteriores.

Palabras clave: Programación Orientada a Objetos, MVC, Scrum, Node.js, Spring Boot y Bases de datos.

Keywords

buenas prácticas, ingeniería de software, investigación aplicada, reproducibilidad

2. Introducción

El desarrollo de software ha experimentado transformaciones significativas en las últimas décadas. La creciente complejidad de los sistemas exige soluciones más organizadas, escalables y seguras. En este escenario, los paradigmas de programación, las metodologías de gestión y las arquitecturas de software se convierten en piezas fundamentales para enfrentar los retos tecnológicos actuales. Según [Trigás Gallego 2012], la ausencia de una arquitectura sólida impacta negativamente en el ciclo de vida del software, dificultando su evolución y mantenibilidad. De igual manera, Gaviláñez et al. [Izquierdo 2008] destacan que los patrones de diseño y la POO garantizan la claridad del código y su reutilización. Por su parte, [Haro et al. 2019] afirman que la adopción de metodologías ágiles como Scrum permite mejorar la comunicación, adaptarse al cambio y entregar valor continuo. En ese sentido, la integración de metodologías y arquitecturas ofrece un marco sólido para la creación de sistemas más eficientes. Este artículo busca integrar los aportes teóricos y prácticos de la POO, el patrón MVC, los frameworks Node.js y Spring Boot, la metodología Scrum y la gestión de bases de datos, con el fin de mostrar su relevancia en el desarrollo de software moderno.

3. Fundamentación Teórica

A. Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos organiza el software en torno a clases y objetos, favoreciendo principios como la abstracción, el encapsulamiento, la herencia y el polimorfismo. Según Introducción a la Programación Orientada a Objetos [Izquierdo 2008], este paradigma promueve la reutilización del código y facilita la modularidad de los proyectos. En comparación con la programación estructurada, la POO ofrece mayor claridad y un modelado más cercano a la lógica del mundo real. La abstracción permite modelar conceptos del entorno dentro del software, mientras que el encapsulamiento protege y mantiene la coherencia de los datos internos. La herencia fomenta la reutilización del código al establecer jerarquías entre clases, y el polimorfismo facilita la extensión de funcionalidades sin alterar estructuras ya establecidas. Además, se resalta el valor de la POO en la enseñanza de la programación, ya que brinda a los estudiantes una forma intuitiva de abordar problemas complejos y mantiene su vigencia en lenguajes como Java, C#, Python o C++ [Izquierdo 2008].

B. Patrón Modelo-Vista-Controlador (MVC)

El patrón Modelo–Vista–Controlador (MVC) divide una aplicación en tres componentes principales: el Modelo (datos y lógica de negocio), la Vista (interfaz de usuario) y el Controlador (intermediario). De acuerdo con Automatización de la codificación del patrón MVC en proyectos orientados a la Web [Camarena Sagredo et al. 2012], esta separación mejora la productividad, la escalabilidad y la consistencia del software, además de facilitar pruebas unitarias y mantenimiento. La propuesta técnica de dicho trabajo presenta una herramienta en Java (GAPRIP MVC) que automatiza la codificación a partir de esquemas de bases de datos en MySQL. El sistema genera clases DTO y DAO, junto con capas de negocio y presentación, lo que reduce tiempos de desarrollo en un 60–70 %. Esto confirma que el patrón MVC, además de organizar el código, permite integrar procesos de automatización que potencian la eficiencia en proyectos web [Camarena Sagredo et al. 2012].

C. Node.js y Spring Boot

Node.js es un entorno ligero y eficiente, ideal para aplicaciones en tiempo real y microservicios, mientras que Spring Boot se caracteriza por su robustez y seguridad en entornos empresariales basados en Java. El estudio Node.js vs Spring Boot en el desarrollo backend de aplicaciones web [Haro et al. 2019] concluye que la elección depende de las necesidades del proyecto: Node.js destaca por su flexibilidad y escalabilidad, mientras que Spring Boot ofrece solidez y estabilidad. En particular, Spring Boot simplifica la construcción de microservicios al proveer autoconfiguración y servidores embebidos, lo que disminuye la complejidad del despliegue. Además, su madurez le permite integrarse con librerías como Spring Security y bases de datos SQL transaccionales. Por otro lado, Node.js, junto con Express.js, aprovecha el modelo asíncrono de JavaScript para ofrecer rapidez y facilidad de integración con bases NoSQL como MongoDB, aunque con ciertas limitaciones en materia de seguridad y dependencia de paquetes externos [Haro et al. 2019].

D. Metodología Scrum

Scrum se basa en iteraciones cortas llamadas Sprints y define roles esenciales como el Scrum Master y el Product Owner. Según Metodología Scrum: Desarrollo detallado de la fase de aprobación de un proyecto informático mediante el uso de metodologías ágiles [Trigás Gallego 2012], este marco de trabajo mejora la organización del equipo, la comunicación con el cliente y la entrega continua de valor. El documento explica cómo los elementos clave —Product Backlog, Sprint Backlog e Incrementos de producto— permiten priorizar requerimientos y entregar software funcional en cada ciclo. Además, su carácter iterativo y adaptable refuerza la capacidad de respuesta frente a cambios del entorno, reduciendo riesgos y aumentando la satisfacción del cliente [Trigás Gallego 2012].

E. Bases de datos

Las bases de datos son el núcleo en la gestión de información. El libro Fundamentos sobre la gestión de base de datos [Pisco Gómez et al. 2017] explica que los modelos relacionales, apoyados en SQL, aseguran consistencia e integridad mediante procesos de normalización y control de dependencias. Por su parte, los modelos NoSQL ofrecen flexibilidad para manejar grandes volúmenes de datos, lo que resulta útil en aplicaciones modernas distribuidas y de rápido crecimiento [Pisco Gómez et al. 2017]. En general, la gestión de bases de datos permite construir sistemas robustos y confiables.

Operaciones como consultas, inserciones y actualizaciones garantizan precisión y disponibilidad, mientras que los mecanismos de control de acceso y llaves primarias y foráneas protegen frente a inconsistencias. Estos principios consolidan a las bases de datos como un componente esencial en cualquier sistema de información.

4. Metodología

Este artículo se desarrolló bajo un enfoque cualitativo de investigación documental, orientado a la revisión y análisis de literatura científica relacionada con paradigmas, metodologías y arquitecturas de software. De acuerdo con Metodología Scrum: Desarrollo detallado de la fase de aprobación de un proyecto informático mediante el uso de metodologías ágiles [Trigás Gallego 2012], los enfoques cualitativos permiten interpretar fenómenos complejos y aportar lineamientos prácticos basados en evidencia académica.

La realización de este proyecto se centró en comprender y aplicar principios clave como los de la Programación Orientada a Objetos [Izquierdo 2008], la arquitectura Modelo–Vista–Controlador [Camarena Sagredo et al. 2012], la gestión de bases de datos [Pisco Gómez et al. 2017] y el uso de metodologías ágiles como Scrum [Trigás Gallego 2012]. Estos lineamientos técnicos y metodológicos habían sido definidos desde las etapas iniciales del proceso formativo como parte del plan de desarrollo de un sistema de gestión de eventos. La revisión documental permitió sustentar dichas decisiones y evidenciar sus beneficios en términos de organización, colaboración y robustez en el desarrollo de software.

Sus fases fueron:

4.1. Fase 1- Recolección y construcción del thesaurus

En esta primera etapa se buscó consolidar un sustento teórico para el proyecto. Para ello, se recurrió a Google Scholar como principal fuente de información, aplicando búsquedas con términos clave vinculados a metodologías ágiles, patrones de diseño, Programación Orientada a Objetos (POO) y el modelo MVC. Los resultados más pertinentes se registraron en un documento tipo thesaurus, en el que se organizaron de manera uniforme datos como el título del artículo, la fuente de publicación, la fecha y el concepto central. El objetivo de esta fase fue recolectar y estructurar evidencias académicas que demostraran la pertinencia de emplear las metodologías y tecnologías seleccionadas dentro del desarrollo del proyecto formativo.

4.2. Fase 2- Análisis cualitativo de la evidencia (síntesis del thesaurus)

La finalidad de esta etapa consistió en convertir la información recopilada en sustento teórico y práctico que respaldara la incorporación de enfoques como el Modelo–Vista–Controlador (MVC), la Programación Orientada a Objetos (POO) y la metodología ágil Scrum en el desarrollo del sistema. Para ello, se llevaron a cabo procesos de lectura analítica, elaboración de resúmenes y contrastación crítica de los artículos seleccionados, lo que permitió identificar elementos clave como la separación de responsabilidades en la arquitectura, la facilidad de mantenimiento del software y el valor del trabajo colaborativo en equipos ágiles.

4.3. Fase 3- Diseño y desarrollo del sistema

El propósito central de esta fase fue llevar a la práctica los fundamentos teóricos y metodológicos previamente analizados, convirtiéndolos en un sistema de gestión de eventos plenamente funcional. Para ello, se tomó como base la arquitectura Modelo-Vista-Controlador (MVC), que facilitó la organización en capas; la Programación Orientada a Objetos (POO), que aportó principios de reutilización y escalabilidad; y la metodología ágil Scrum, que orientó el proceso de construcción mediante una gestión iterativa y colaborativa. En conjunto, estas guías permitieron estructurar y coordinar el proyecto de manera ordenada, asegurando tanto la calidad técnica del software como la efectividad en la organización del trabajo del equipo.

5. Diseño y desarrollo del sistema

El propósito central de esta fase fue llevar a la práctica los fundamentos teóricos y metodológicos previamente analizados, convirtiéndolos en un sistema de gestión de eventos plenamente funcional. Para ello, se tomó como base la arquitectura Modelo-Vista-Controlador (MVC), que facilitó la organización en capas; la Programación Orientada a Objetos (POO), que aportó principios de reutilización y escalabilidad; y la metodología ágil Scrum, que orientó el proceso de construcción mediante una gestión iterativa y colaborativa. En conjunto, estas guías permitieron estructurar y coordinar el proyecto de manera ordenada, asegurando tanto la calidad técnica del software como la efectividad en la organización del trabajo del equipo.

6. Resultados

Los hallazgos obtenidos evidencian lo siguiente: Estos resultados refuerzan la importancia de integrar metodologías y arquitecturas sólidas en el desarrollo moderno de software.

6.1. Programación Orientada a Objetos (POO) y Modelo-Vista-Controlador (MVC)

Ambos enfoques aportan modularidad y escalabilidad al desarrollo de software, ya que favorecen la separación de responsabilidades y la organización del código [Izquierdo 2008][Camarena Sagredo et al. 2012]. En este proyecto se utilizó Java como lenguaje principal, aprovechando sus características orientadas a objetos para estructurar de manera clara y adaptable los componentes del sistema.

6.2. Frameworks backend (Node.js y Spring Boot)

Node.js destaca por su eficiencia en aplicaciones de microservicios y entornos que requieren alta escalabilidad, mientras que Spring Boot ofrece mayor solidez y seguridad en proyectos empresariales [Haro et al. 2019]. En el caso del presente trabajo, se optó por emplear Spring Boot debido a su facilidad de configuración y a las ventajas que proporciona en entornos corporativos. Aunque no se implementó bajo un modelo de microservicios, se desarrolló una arquitectura monolítica bien estructurada que permitió cumplir con los objetivos formativos del sistema.

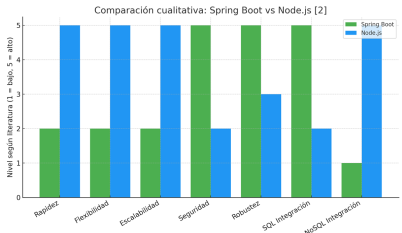


Figura 1: Comparación entre Node.js y Spring Boot en el desarrollo backend.

6.3. Metodología Scrum

Este marco de trabajo fortalece la comunicación y la colaboración en equipo, además de facilitar la entrega incremental de valor [Trigás Gallego 2012]. En el proyecto no se adoptó Scrum en su totalidad, pero se aplicaron prácticas inspiradas en esta metodología, como la organización de tareas mediante un modelo tipo Canva para estructurar los Sprints y dar seguimiento a los avances de manera iterativa.



Figura 2: Implementación de la metodología Scrum en el proyecto.

Y de igual manera implementamos los dailys en llamadas en Discord.

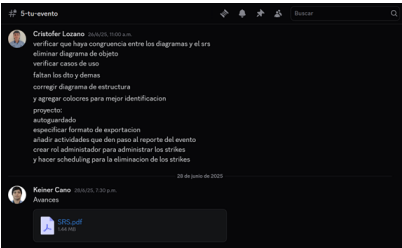


Figura 3: Comunicación diaria en Discord para el seguimiento del proyecto.

6.4. Las bases de datos

Las bases de datos relacionales continúan siendo esenciales, aunque las NoSQL amplían el espectro en escenarios, para nuestro proyecto el uso de Fetch es útil para las comunicaciones y usamos NoSQL en los entornos móviles y web, para hacer comunicación

entre la API y la base de datos haciendo cuerpos JSON desde el Frontend, para el proyecto de eventos usamos una base de datos con el motor SQL de PostgreSQL.

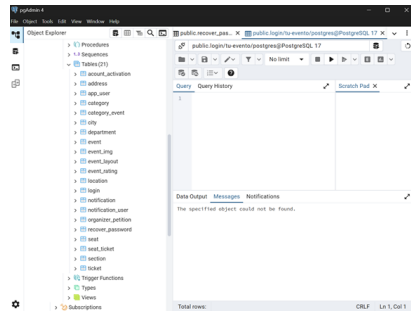


Figura 4: Estructura de la base de datos PostgreSQL utilizada en el proyecto.

Estos resultados refuerzan la importancia de integrar metodologías y arquitecturas sólidas en el desarrollo moderno de software.

7. Discusión

Interpretamos efectos, generalización y costos/beneficios. Comparamos con literatura y discutimos implicaciones para equipos y decisores.

8. Conclusión

El análisis documental confirma que la integración de paradigmas de programación, arquitecturas de software y metodologías ágiles constituye un eje fundamental en el desarrollo contemporáneo de sistemas. Según Introducción a la Programación Orientada a Objetos [Izquierdo 2008], la POO facilita modularidad y adaptabilidad en el código, mientras que el patrón Modelo–Vista–Controlador (MVC) aporta organización y escalabilidad en las aplicaciones [Camarena Sagredo et al. 2012]. Por su parte, Node.js vs Spring Boot en el desarrollo backend de aplicaciones web [Haro et al. 2019] evidencia que la elección de frameworks debe responder a las necesidades específicas del proyecto, equilibrando rapidez y flexibilidad frente a seguridad y robustez.

Asimismo, la Metodología Scrum [Trigás Gallego 2012] refuerza la comunicación del equipo, la planificación y la entrega incremental de valor, y el libro Fundamentos sobre la gestión de bases de datos [Pisco Gómez et al. 2017] resalta la importancia de modelos sólidos para garantizar la consistencia e integridad de la información.

En conjunto, la combinación de la Programación Orientada a Objetos, la arquitectura MVC, frameworks modernos como Node.js y Spring Boot, metodologías ágiles y bases de datos robustas permite construir sistemas más escalables, seguros y alineados con las demandas actuales de la industria del software.

A. Agradecimientos

El presente trabajo no habría sido posible sin el apoyo brindado por el Servicio Nacional de Aprendizaje (SENA), que proporcionó los recursos académicos y formativos necesarios para la realización de este proyecto. De igual manera, se agradece al instructor Jesús

Ariel González Bonilla, por su guía, orientación y constante acompañamiento en cada etapa del proceso.

Finalmente, se extiende un especial agradecimiento a las fuentes académicas y científicas consultadas, que ofrecieron el sustento teórico y metodológico indispensable para fortalecer la investigación y garantizar la validez de los resultados obtenidos.

B. Conflictos de interés / Declaración ética

Los autores declaran que no existen conflictos de interés relacionados con la elaboración del presente artículo. El desarrollo de esta investigación se realizó exclusivamente con fines académicos y formativos, respetando en todo momento la integridad intelectual y ética en el uso de fuentes bibliográficas.

C. Contribuciones de los autores (CRediT)

- Redacción – borrador original: Keiner Andrés Cano Narvárez
- Redacción – revisión y edición: Keiner Andrés Cano Narvárez
- Visualización (imágenes, grafica): Keiner Andrés Cano Narvárez
- Supervisión académica: Instructor Jesús Ariel González Bonilla

Referencias

- Jesús Gamaliel Camarena Sagredo, Adrian Trueba Espinosa, Magally Martínez Reyes, and María de Lourdes López García. 2012. Automatización de la codificación del patrón modelo-vista-controlador (MVC) en proyectos orientados a la Web. <https://www.redalyc.org/pdf/104/10423895005.pdf>
- Edward Haro, Teresa Guarda, Alex Omar Zambrano Peñaherrera, and Geovanni Ninahualpa Quiña. 2019. Desarrollo backend para aplicaciones web, Servicios Web Restful: Node.js vs Spring Boot. <https://www.proquest.com/openview/a78cfaa62708fd24f38ac8d1025050eb/1?pq-origsite=gscholar&cbl=10063>
- Luis R. Izquierdo. 2008. Introducción a la Programación Orientada a Objetos. <https://luis.izqui.org/resources/ProgOrientadaObjetos.pdf>
- Ángel Pisco Gómez, Julio Johnny Regalado Jalca, Jimmy Gutiérrez García, Omar Quimis Sánchez, Kleber Marcillo Parrales, and Javier Marcillo Merino. 2017. Fundamentos sobre la gestión de base de datos. https://books.google.com.co/books?hl=es&lr=lang_es&id=H0VBDwAAQBAJ&oi=fnd&pg=PA21
- Manuel Trigás Gallego. 2012. Metodología Scrum: Desarrollo detallado de la fase de aprobación de un proyecto informático mediante el uso de metodologías ágiles. <https://openaccess.uoc.edu/server/api/core/bitstreams/64de036d-c56c-4317-ad60-fcb968469221/content>